

M-Agent 生产 Runtime 层实施计划

蓝色章节条用于定位结构；黄色提示框表示结论、边界或风险；表格中的加粗项是当前决策重点。

源文档：production-runtime-layer-plan.zh-CN.md · 状态日期：2026-07-30

生产 Runtime 层实施计划

状态快照：2026-07-30 (**P8 验收矩阵已完成**；**RuntimeHost + LangGraphRuntime 已落地**；**R1 内部验证已通过**；**R2 graph 内 thinking→delegate→feedback 闭环已通过**；Chat API 仍仅接 ThinkLife)

范围：把 acceptance 已证明的双 Runtime 语义，接到可承载真实对话的生产 Runtime 层与 Chat API 灰度入口。

本文回答三个问题：

- P8 之后「生产 Runtime 层」与 acceptance 层差在哪里；
- 应按什么顺序把 LangGraph 接到真实流量；
- 每一阶段的产出、验收门与回退方式是什么。

阶段定位

官方 P0 ~ P8 已在 **验收平台** 收口。当前实质阶段为 **P8 后 • 生产 Runtime 接线期**：

目标不是再扩测试矩阵，而是让 **HTTP / 内部 thread** 能按 `runtime_engine` 稳定执行。

1. 当前位置

1.1 已完成

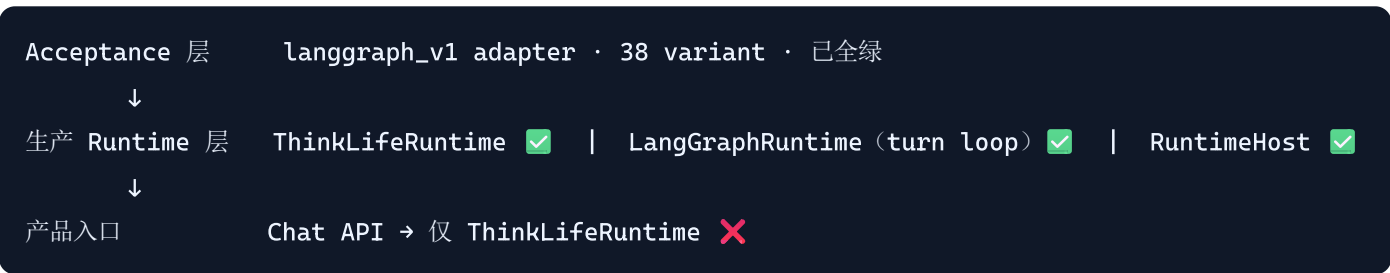
层级	状态	说明
设计总线与 P1 契约	✓	TX/SP/AT 语义、38 variant、双 Runtime binding
ThinkLife 生产 Runtime	✓	<code>ThinkLifeRuntime</code> ；Chat API 全量使用
P8 acceptance LangGraph	✓	38/38 variant；持久 SQLite checkpointer； <code>routing.py</code>
稳定观察期（首轮）	✓	双 Runtime 全矩阵各 3 轮 + Robustness 重点切片全绿
RuntimeHost 协议	✓	<code>src/m_agent/runtime/host/</code>
LangGraphRuntime MVP	✓	<code>src/m_agent/runtime/langgraph/runtime.py</code> ；最小 inbox→graph 链路
R1 内部 thread 验证	✓	<code>scripts/smoke_langgraph_runtime.py</code> ；5 轮 × 116 检查全绿

层级	状态	说明
R2 graph 内完整循环	✅	turn_graph.py ; scripts/smoke_langgraph_turn_loop.py 3 轮 × 78 检查全绿

1.2 尚未完成

缺口	影响
delegate 默认仍为 fake capability	真实工具需 delegate_executor=execution_agent ; execution 层尚未按 runtime_engine 分发
Chat API 按 runtime_engine 分发	用户流量 100% ThinkLife; 设 env 可能「标注与执行不一致」
生产灰度与旧 loop 收缩	文档规划的最后一步，尚未开始

1.3 三层结构（当前）



关键边界

- Transaction Store 仍是 WM/任务状态/四态的权威；LangGraph checkpoint 只保存图执行恢复态。
- 已开始运行的 transaction 不在中途切换 Runtime；回退只影响 后续新建 transaction 的默认路由。

2. 目标与成功标准

2.1 总目标

在 不破坏 ThinkLife 现有用户路径 的前提下，让 LangGraph 成为可选、可灰度、可回退的生产 transaction 执行引擎。

2.2 阶段成功标准（汇总）

里程碑	完成条件
R1 内部验证	非 acceptance script 下，LangGraphRuntime 跑通 fake 对话链路
R2 完整 graph 循环	thinking → delegate → feedback → Scene 在 LangGraph host 内闭环
R3 Chat API 分发	HTTP 入口按 runtime_engine 选择 RuntimeHost; health 可观测
R4 生产灰度	内部 thread → 默认 LangGraph; 可一键回退 ThinkLife

里程碑	完成条件
R5 旧 loop 收缩	无非终态旧 Runtime transaction; 观察期后删除/归档旧循环

3. 架构要点

3.1 RuntimeHost (engine-neutral 门面)

位置： `src/m_agent/runtime/host/`

组件	职责
<code>protocol.py</code>	RuntimeHost 协议: <code>submit / run_thread / health / shutdown</code>
<code>think_life_adapter.py</code>	包装现有 <code>ThinkLifeRuntime</code>
<code>factory.py</code>	<code>create_runtime_host(agent, runtime_engine=...)</code>

Chat API 与未来 orchestrator 应依赖 **RuntimeHost**，而不是直接依赖具体 engine 类。

3.2 LangGraphRuntime (生产引擎)

位置： `src/m_agent/runtime/langgraph/`

组件	职责
<code>runtime.py</code>	生产 host: <code>Store / Inbox / Gateway / Attributor / Scene / Drainer</code>
<code>inbox_loop.py</code>	drain: 归因 → turn graph (或回退 MVP graph step) → Scene
<code>engine.py</code> + <code>checkpointer.py</code>	持久 graph 执行与 SQLite checkpoint
<code>transaction_graph.py</code>	MVP StateGraph (<code>record_progress</code> , R2 回退目标)
<code>turn_graph.py</code>	R2 Turn StateGraph: <code>think → plan_delegate → delegate → settle_turn</code>
<code>turn_ports.py</code>	出边口: <code>TurnPlanner / DelegateExecutor / DelegateEffectLedger</code>
<code>config.py</code>	engine 本地开关 (turn loop、delegate executor、delivery guarantee)

持久化 (按 user)：

- `{persist_root}/runtime/langgraph.sqlite3` — Transaction Store
- `{persist_root}/runtime/langgraph-checkpoints.sqlite3` — MVP graph checkpoint
- `{persist_root}/runtime/langgraph-turn-checkpoints.sqlite3` — turn graph checkpoint

两张 checkpoint 表**分文件、分 thread 命名空间** (turn graph 用 `turn:{transaction_id}`)，避免 state schema 不同的两张图互相覆盖。

新建 transaction 默认 `runtime_engine=langgraph_v1` (`TransactionRegistry.default_runtime_engine`)。

3.3 灰度路由

位置: `src/m_agent/runtime/routing.py`

- 创建 transaction 时固定 `runtime_engine`
- `M_AGENT_DEFAULT_RUNTIME_ENGINE=langgraph_v1|think_life_v1` 控制 **新 transaction** 默认归属
- 回退 = 改 env, **不影响** 进行中 transaction

R3 之前禁止

在 Chat API 生产环境开启 `M_AGENT_DEFAULT_RUNTIME_ENGINE=langgraph_v1`, 除非 execution 层已按 `runtime_engine` 分发; 否则 Store 标注与真实执行会不一致。

4. 实施阶段 (R1 ~ R5)

R1: 内部 thread 验证 (优先, 约 1~2 天) ☒ 已完成

目标: 证明 LangGraphRuntime 在 **非 acceptance** 路径可稳定运行。

任务:

1. ☒ 新增 `scripts/smoke_langgraph_runtime.py` (或同等 dev 入口)
2. ☒ 使用 `create_runtime_host(..., runtime_engine=langgraph_v1)`
3. ☒ 覆盖: `submit_user_message` → `run_thread` → 检查 Store / Scene / checkpoint
4. ☒ 可选: 进程重启后 checkpoint + Store 一致性抽查

退出条件:

- ☒ 脚本可重复运行 5 次无失败 (`--rounds 5`, 5/5 通过, 每轮 116 项检查)
- ☒ `runtime_engine`、WM、revision 与 graph_phase 一致 (Store × checkpoint 逐 transaction 对齐)

运行方式:

```
python scripts/smoke_langgraph_runtime.py --rounds 5
python scripts/smoke_langgraph_runtime.py --rounds 5 --verbose --json-out .tmp/r1.json
```

脚本使用 stub ThinkingAgent (无模型调用), 每轮独立 persist_root; stdout 为 JSON 汇总, 进度行走 stderr; 全绿退出码 0, 否则 1。 `tests/runtime/test_smoke_langgraph_runtime.py` 以单轮跑同一入口, 防止脚本随实现漂移。

回退: 仅 dev 脚本, 不影响 Chat API。

R2: LangGraph 内完整循环 (核心, 约 1~2 周) ☒ 已完成

目标: 用真实 Thinking + delegate + Feedback 替代 MVP 的 `record_progress` 占位。

任务:

- 1. ☒ 新增 `turn_graph.py` 并由 `LangGraphInboxLoop` 驱动:
 - ☒ `think` 节点调 `ThinkingAgent.handle` , 复用 `ThinkLife` 的 `think_context / WM / Scene tail`
 - ☒ `plan_delegate` 节点把决策落成 `delegate intent` (含 `reply_to_user` 归一化)
 - ☒ `delegate` 节点执行并把结构化 `Feedback` 经 `Gateway` 送回 `inbox`, 下一次 `drain resume graph`
 - ☒ `settle_turn` 节点按 `revision fence` 落 `Store`, 并写回 `checkpoint`
- 2. ☒ 复用 `P7` 外层: `EffectCoordinator` 登记 `effect intent`、`Feedback outbox`、`capability delivery guarantee`
- 3. ☒ 对照 `acceptance TX-07` 切片做内部回归 (契约未改)

关键设计:

主题	处理
一次 drain = 一个 turn	graph 不在节点内自旋等 <code>Feedback</code> ; <code>Feedback</code> 作为新 <code>stimulus</code> 触发下一 <code>turn</code> , 故 <code>host</code> 重启可续跑
状态权威	<code>Store</code> 仍是唯一权威; graph 每次提交带 <code>expected revision (RuntimeUnitOfWork)</code>
delegate 链上限	<code>max_delegate_chain</code> (默认 32) , 超限即 <code>settle</code> , 避免自触发死循环
Scene 因果	<code>thought / action / reply</code> 均绑定 <code>transaction_id + delegate_id</code> , <code>seq</code> 严格递增
USER_TASK 收口	<code>turn</code> 结束不 <code>COMPLETE</code> , 交给 <code>flush</code> ; 非 <code>USER_TASK</code> 当轮闭环

退出条件:

- ☒ 内部 `thread` 完成: 用户消息 → 思考 → `fake tool` → 结构化 `Feedback` → `Scene` → `reply`
- ☒ 与 `ThinkLife` 同输入、同 `scripted` 决策下, **领域观察面** (`transaction` 四态/状态、`delegate status`、`stimulus disposition`、`Scene` 序列) 逐字段相同
- ☒ `host` 重启后 `Store × Scene × turn checkpoint` 三方一致 (`revision / WM / graph_phase`)
- ☒ 回退开关关闭后行为退回 `R1 record_progress` , 且 `ThinkingAgent` 一次未被调用

运行方式:

```
python scripts/smoke_langgraph_turn_loop.py --rounds 3
python scripts/smoke_langgraph_turn_loop.py --rounds 3 --verbose --json-out .tmp/r2.json
python scripts/smoke_langgraph_turn_loop.py --rounds 3 --skip-compare
```

脚本使用 `stub Thinking/Execution Agent` (无模型调用) , 一个入口同时跑两道门: **turn loop 冒烟** (含 `host` 重启与回退验证) 与 **ThinkLife 领域对比抽样**。每轮 78 项检查, `tests/runtime/test_langgraph_turn_loop.py` 以单测覆盖同一批不变量防漂移。

回退: `M_AGENT_LANGGRAPH_TURN_LOOP=0` , 或 `agent config runtime.langgraph.turn_loop=false` , 即回退 MVP `record_progress` 路径 (`turn_engine=None` , `health()["turn_loop"]=false`) 。优先级与 `resolve_runtime_engine` 一致: 显式 `config` > `env` > 默认开。

R3: Chat API RuntimeHost 分发 (约 3~5 天, 依赖 R2)

目标: HTTP 入口不再硬编码 `ThinkLifeRuntime`。

任务:

1. `chat_api_runtime.py` 持有 `RuntimeHost` (或按 `thread/transaction` 解析 engine)
2. `submit_user_message` / `run_thread` / 相关 health 走 host 协议
3. 保留 ThinkLife 专用 `flush/schedule` 扩展的过渡适配 (可先仅在 ThinkLife host 暴露)
4. `snapshot` / `health` 增加 `runtime_engine_id` 与 per-engine 指标

退出条件:

- 内部 conversation 可通过 Chat API 走 LangGraph
- 默认仍为 ThinkLife; 显式 config 可切 LangGraph
- 单 transaction 全生命周期 engine 不变

回退: 配置默认 engine 切回 `think_life_v1`; 进行中 LangGraph transaction 继续由 LangGraph 完成。

R4: 生产灰度 (约 1~2 周观察, 依赖 R3)

顺序 (与 P8 文档一致) :

1. fake tool 环境
2. 内部 conversation
3. 新建普通 user transaction
4. Scheduled Plan transaction
5. 默认新 transaction 路由 LangGraph
6. 稳定观察期

任务:

1. 部署侧配置 `M_AGENT_DEFAULT_RUNTIME_ENGINE` 分批 rollout
2. 每日/每次发布跑观察期脚本 (双 Runtime 全矩阵 + Robustness)
3. 监控: 重复 effect、Scene/flush 漂移、checkpoint 与 Store revision 冲突

退出条件:

- 灰度期间无未登记 semantic failure
 - 无「Store 与 checkpoint 静默覆盖」incident
 - 回退演练通过 (env 切回 ThinkLife, 新 transaction 不再进 LangGraph)
-

R5: 旧 Runtime 收缩 (最后)

前置: R4 观察期结束。

任务:

1. 清点非终态、可恢复 archive 的 ThinkLife-only transaction
2. 版本化迁移或只读兼容层
3. 删除/归档 ThinkLife transaction 内循环中已被 LangGraph 替代的路径

退出条件：

- 不存在只能由即将删除代码恢复的非终态 transaction
- 文档与运维 runbook 更新

5. 测试与观察策略

5.1 必跑门禁

频率	命令 / 范围
每次 PR	<code>think_life_v1</code> + <code>langgraph_v1</code> acceptance scenarios; <code>pytest tests/runtime</code>
每日 / 发布前	全矩阵 3 轮 + Robustness 重点 7 切片（观察期脚本）
R1 后	<code>python scripts/smoke_langgraph_runtime.py --rounds 5</code>
R2 后	<code>python scripts/smoke_langgraph_turn_loop.py --rounds 3</code> （内部 smoke + TX-07 领域对比抽样）
R3 后	Chat API integration 双 engine smoke

5.2 建议固化脚本

将下列流程固化为 `scripts/run_observation_period.ps1` （或 CI job）：

- 双 Runtime 全矩阵 × N 轮
- Robustness 关键字切片
- 输出 pass/fail 汇总 JSON

6. 风险与回退

风险	控制
Store 标 LangGraph、实际跑 ThinkLife	R3 前禁止生产 env 默认 LangGraph; execution 与 <code>runtime_engine</code> 对齐后再开
checkpoint 覆盖 Store 权威	所有 graph 提交带 expected revision; Pause/Delete 后旧 checkpoint 不得 silent reload
turn graph 与 MVP graph checkpoint 串味	两图分 checkpoint 文件; turn graph thread 命名空间为 <code>turn:{transaction_id}</code>
Feedback 自触发死循环	<code>max_delegate_chain</code> 上限后强制 settle; 每轮 delegate 必须匹配 pending intent 才推进
双 engine Scene/flush 漂移	共享 Scene Store; flush 仍走统一 FlushCoordinator
灰度 incident	只回退 新 transaction 默认路由; 保留 audit 与 engine 字段

7. 与现有文档的关系

文档	关系
current-project-progress-and-design-plan.md	P0 ~ P8 已完成；本计划是 P8 后的生产接线专篇
langgraph-runtime-migration-plan.zh-CN.md	状态权威与 Runtime Host 总线；本计划是其生产落地时间表
semantic-acceptance-platform.zh-CN.md	验收平台用法；R1 ~ R5 仍以 38 variant 为回归门禁

冲突时优先级：

设计总线 → 共享语义契约 → P8 阶段结论 → 本生产 Runtime 计划 → 具体实现细节

8. 当前立即执行的下一步

- 1. ~~R1：编写并跑通 smoke_langgraph_runtime 内部脚本（5 次重复）~~ ☒ 已完成
- 2. ~~R2：启动 graph 内 thinking/delegate/feedback 循环（最大块）~~ ☒ 已完成
- 3. R3：Chat API 改持 RuntimeHost，按 runtime_engine 分发
- 4. 并行：execution 层按 runtime_engine 分发，使 delegate_executor=execution_agent 可上生产
- 5. 并行：观察期脚本 CI 化（不阻塞 R3）

不在此阶段做：Chat API 默认 LangGraph、删除 ThinkLife loop。

9. 文件索引（实现落点）

路径	说明
src/m_agent/runtime/host/	RuntimeHost 协议与 factory
src/m_agent/runtime/langgraph/runtime.py	LangGraphRuntime 生产 host
src/m_agent/runtime/langgraph/inbox_loop.py	生产 inbox drain（turn loop / MVP 二选一）
src/m_agent/runtime/langgraph/turn_graph.py	R2 turn StateGraph 与 TransactionTurnEngine
src/m_agent/runtime/langgraph/turn_ports.py	planner / delegate executor / effect ledger 出边口
src/m_agent/runtime/langgraph/config.py	LangGraphRuntimeConfig 与 turn loop 回退开关
src/m_agent/runtime/routing.py	灰度默认路由
src/m_agent/api/chat_api_runtime.py	Chat API（R3 改造入口）

路径	说明
scripts/smoke_langgraph_runtime.py	R1 内部 thread 冒烟入口
scripts/smoke_langgraph_turn_loop.py	R2 turn loop 冒烟 + ThinkLife 领域对比入口
tests/runtime/test_runtime_host.py	Host MVP 单测 (含 turn loop 回退)
tests/runtime/test_smoke_langgraph_runtime.py	R1 冒烟脚本防漂移单测
tests/runtime/test_langgraph_turn_loop.py	R2 turn loop 不变量单测
tests/acceptance/scenarios/	38 variant 回归门禁